Java: Payment Validation and Resource Management

Hard

Overview

Implement a system to process and validate payment information. The goal is to ensure that the payment data is correct and that resources are managed (using try-with-resources) efficiently. The system has three classes. The first class (PaymentInfo) has been provided, while the other two (PaymentValidationService and PaymentReaderService) must be implemented: The application receives a list of string commands[5], where each line represents a specific detail of the payment. The expected input format is as follows: Example Input Output Explanation Class Description Complete the class PaymentValidationService in the editor below. Methods Complete the class PaymentReaderService in the editor below. Methods The following exception classes must be implemented: Constraints The application will receive a list of string commands, where each string represents a specific input parameter: Sample Input 0 Sample Output 0 Explanation Sample Input 1 Sample Output 1 Explanation The PaymentInfo class represents the details of a payment transaction. It encapsulates information such as the payment date, amount, payer and receiver account numbers, and the purpose of the payment. The PaymentReaderService class reads and parses payment details to construct a PaymentInfo object and ensures proper resource management. The PaymentValidationService class validates the details of a PaymentInfo object and ensures that the payment data is correct. It also manages resources and ensures proper cleanup.Date of Payment: The payment date in dd-mm-yyyy format.Amount: The payment amount is a positive number.Payer Account Number: The account number of the payer.Receiver Account Number: The account number of the receiver.Purpose of Payment: The reason for the payment.The read method of PaymentReaderService class reads the inputs and successfully parses the date (12-07-2024) and amount (250.00). It also reads the payer account number (1234567890), receiver account number (0987654321), and purpose of payment ("Rent Payment").A PaymentInfo object is created with these details.The validate method of PaymentValidationService class validates the payment details: * The amount is positive (250.00 > 0). * Payer and receiver account numbers are valid (not empty and different). * The purpose of payment is provided. No exceptions are thrown, and the resources (PaymentReaderService and PaymentValidationService) are closed with appropriate messages written to the bufferedWriter. PaymentValidationService(BufferedWriter bufferedWriter): Initializes the service with a bufferedWriter to log information about the service's lifecycle. validate(PaymentInfo payment): Validates the provided payment object. * Checks if the payment amount is positive; if not, it throws an InvalidPaymentDetailsException with the message "Amount must be greater than zero. Provided: " followed by the actual amount. * Checks if the payer and receiver account numbers are not empty and not the same; if not, it throws an InvalidPaymentDetailsException with the message "Payer and receiver accounts are invalid. Payer: " followed by the payer account number and ", Receiver: " followed by the receiver account number. * Checks if the purpose of payment is not empty; if not, it throws an InvalidPaymentDetailsException with the message "Purpose of payment is missing." close(): Closes the resources associated with this service. Prints the message "PaymentValidationService. Resource closed" using the bufferedWriter. This method should be called automatically when the try-with-resources block is exited. PaymentReaderService(BufferedWriter bufferedWriter): Initializes the service with a bufferedWriter for logging messages related to the service's lifecycle. read(List<String> inputs) -> PaymentInfo: Parses and validates the provided list of inputs to create a PaymentInfo object. * Checks if the inputs contain at least five lines; if not, it throws a MissingPaymentDetailException with the message "Not enough input details provided." * Validates that the date is in the dd-mm-yyyy format, throwing InvalidDateFormatException if the date format is incorrect. * Validates that the amount is a valid positive Double number, throwing InvalidAmountFormatException if the amount format is invalid. close(): Prints the message "PaymentReaderService. Resource closed" using the bufferedWriter. This method should be called automatically when the try-with-resources block is exited. InvalidDateFormatException(String date): The constructor takes a String parameter representing the invalid date and includes it in the error message. Example message: "Invalid date format for date: 31-02-2024". InvalidAmountFormatException(String amount): The constructor takes a String parameter representing the invalid amount and includes it in the error message. Example message: "Invalid amount format for amount: abc". MissingPaymentDetailException(String detail): The constructor takes a String

parameter representing the missing detail and includes it in the error message. Example message: "Missing payment detail: amount". InvalidPaymentDetailsException(String message): The constructor takes a String parameter representing the specific validation failure and includes it in the error message. Example message: "Invalid payment details: Amount must be greater than zero. Provided: 0". n = 5, where n is the size of the array commands 0 < the length of Date of Payment ≤ 100 < the length of Amount ≤ 100 < the length of Payer Account Number ≤ 100 < the length of Receiver Account Number ≤ 100 < the length of Purpose of Payment ≤ 100Date of PaymentAmountPayer Account NumberReceiver Account NumberPurpose of PaymentThe read method of the PaymentReaderService class reads the inputs: * It tries to parse the date (2023/07/31) but fails because the format is incorrect (it should be dd-mm-yyyy). * This triggers an InvalidDateFormatException. An InvalidDateFormatException is thrown with the message "Invalid date format for date: 2023/07/31." No validation of payment details occurs because the exception is thrown during the date parsing step.The resources (PaymentReaderService and PaymentValidationService) are closed with appropriate messages written to the bufferedWriter.The exception is caught in the try block in the main method, and the exception message is written to the bufferedWriter: "InvalidDateFormatException : Invalid date format for date: 2023/07/31".The read method of PaymentReaderService class reads the inputs and successfully parses the date (31-07-2023) and amount (-100.00). It also reads the payer account number (1234567890), receiver account number (0987654321), and purpose of payment ("Payment for services").A PaymentInfo object is created with these details.The validate method of PaymentValidationService class validates the payment details: * The amount is not positive (-100.00 <= 0). * An InvalidPaymentDetailsException is thrown with the message "Amount must be greater than zero. Provided: -100.00" The resources (PaymentReaderService and PaymentValidationService) are closed with appropriate messages written to the bufferedWriter.The exception is caught in the try block in the main method, and the exception message is written to the bufferedWriter: "InvalidPaymentDetailsException : Amount must be greater than zero. Provided: -100.00"

Java (Advanced)
Coding
100 score
40 mins
16 test cases
Java, Exceptions

Question

Implement a system to process and validate payment information. The goal is to ensure that the payment data is correct and that resources are managed (using try-with-resources) efficiently.

The system has three classes. The first class (*PaymentInfo*) has been provided, while the other two (*PaymentValidationService* and *PaymentReaderService*) must be implemented:

- The *PaymentInfo* class represents the details of a payment transaction. It encapsulates information such as the payment date, amount, payer and receiver account numbers, and the purpose of the payment.
- The *PaymentReaderService* class reads and parses payment details to construct a *PaymentInfo* object and ensures proper resource management.
- The *PaymentValidationService* class validates the details of a *PaymentInfo* object and ensures that the payment data is correct. It also manages resources and ensures proper cleanup.

The application receives a list of string *commands[5]*, where each line represents a specific detail of the payment. The expected input format is as follows:

- Date of Payment: The payment date in dd-mm-yyyy format.
- Amount: The payment amount is a positive number.

- Payer Account Number: The account number of the payer.
- Receiver Account Number: The account number of the receiver.
- Purpose of Payment: The reason for the payment.

Example

Input

```
12-07-2024
250.00
1234567890
0987654321
Rent Payment
```

Output

```
PaymentValidationService. Resource closed
PaymentReaderService. Resource closed
```

Explanation

- The *read* method of *PaymentReaderService* class reads the inputs and successfully parses the date (12-07-2024) and amount (250.00). It also reads the payer account number (1234567890), receiver account number (0987654321), and purpose of payment ("Rent Payment").
- A *PaymentInfo* object is created with these details.
- The *validate* method of *PaymentValidationService* class validates the payment details:
    - The amount is positive (250.00 > 0).
    - Payer and receiver account numbers are valid (not empty and different).
    - The purpose of payment is provided.
- No exceptions are thrown, and the resources (*PaymentReaderService* and *PaymentValidationService*) are closed with appropriate messages written to the *bufferedWriter*.

Class Description

Complete the class *PaymentValidationService* in the editor below.

Methods

- *PaymentValidationService*(BufferedWriter *bufferedWriter*): Initializes the service with a *bufferedWriter* to log information about the service's lifecycle.
- *validate*(PaymentInfo *payment*): Validates the provided *payment* object.
    - Checks if the *payment* amount is positive; if not, it throws an *InvalidPaymentDetailsException* with the message "Amount must be greater than zero. Provided: " followed by the actual amount.

    - Checks if the payer and receiver account numbers are not empty and not the same; if not, it throws an *InvalidPaymentDetailsException* with the message "Payer and receiver accounts are invalid. Payer: " followed by the payer account number and ", Receiver: " followed by the receiver account number.
    - Checks if the purpose of *payment* is not empty; if not, it throws an *InvalidPaymentDetailsException* with the message "Purpose of payment is missing."
- *close*(): Closes the resources associated with this service. Prints the message "PaymentValidationService. Resource closed" using the *bufferedWriter*. This method should be called automatically when the try-with-resources block is exited.

Complete the class *PaymentReaderService* in the editor below.

Methods

- *PaymentReaderService*(BufferedWriter *bufferedWriter*): Initializes the service with a *bufferedWriter* for logging messages related to the service's lifecycle.
- *read*(List<String> *inputs*) -> *PaymentInfo*: Parses and validates the provided list of *inputs* to create a *PaymentInfo* object.
    - Checks if the *inputs* contain at least five lines; if not, it throws a *MissingPaymentDetailException* with the message "Not enough input details provided."
    - Validates that the date is in the dd-mm-yyyy format, throwing *InvalidDateFormatException* if the date format is incorrect.
    - Validates that the amount is a valid positive Double number, throwing *InvalidAmountFormatException* if the amount format is invalid.
- *close*(): Prints the message "PaymentReaderService. Resource closed" using the *bufferedWriter*. This method should be called automatically when the try-with-resources block is exited.

The following exception classes must be implemented:

- *InvalidDateFormatException*(String *date*): The constructor takes a String parameter representing the invalid *date* and includes it in the error message. Example message: "Invalid date format for date: 31-02-2024".
- *InvalidAmountFormatException*(String *amount*): The constructor takes a String parameter representing the invalid *amount* and includes it in the error message. Example message: "Invalid amount format for amount: abc".
- *MissingPaymentDetailException*(String *detail*): The constructor takes a String parameter representing the missing detail and includes it in the error message. Example message: "Missing payment detail: amount".
- *InvalidPaymentDetailsException*(String *message*): The constructor takes a String parameter representing the specific validation failure and includes it in the error message. Example message: "Invalid payment details: Amount must be greater than zero. Provided: 0".

Constraints

- $n = 5$, where n is the size of the array *commands*
- $0 <$ the length of *Date of Payment* ≤ 10
- $0 <$ the length of *Amount* ≤ 10
- $0 <$ the length of *Payer Account Number* ≤ 10
- $0 <$ the length of *Receiver Account Number* ≤ 10
- $0 <$ the length of *Purpose of Payment* ≤ 100

Input Format for Custom Testing

The application will receive a list of string *commands*, where each string represents a specific input parameter:

- Date of Payment
- Amount
- Payer Account Number
- Receiver Account Number
- Purpose of Payment

Sample Case 0

Sample Input 0

```
2023/07/31
100.00
1234567890
0987654321
Payment for services
```

Sample Output 0

```
PaymentValidationService. Resource closed
PaymentReaderService. Resource closed
InvalidDateFormatException : Invalid date format for date: 2023/07/31
```

Explanation

- The *read* method of the *PaymentReaderService* class reads the inputs:
    - It tries to parse the date (2023/07/31) but fails because the format is incorrect (it should be dd-mm-yyyy).
    - This triggers an *InvalidDateFormatException*.
      An *InvalidDateFormatException* is thrown with the message "Invalid date format for date: 2023/07/31."
- No validation of payment details occurs because the exception is thrown during the date parsing step.
- The resources (*PaymentReaderService* and *PaymentValidationService*) are closed with appropriate messages written to the *bufferedWriter*.
- The exception is caught in the try block in the main method, and the exception message is written to the *bufferedWriter*: "InvalidDateFormatException : Invalid date format for date: 2023/07/31".

Sample Case 1

Sample Input 1

```
31-07-2023
```

```
-100.00
1234567890
0987654321
Payment for services
```

Sample Output 1

```
PaymentValidationService. Resource closed
PaymentReaderService. Resource closed
InvalidPaymentDetailsException : Invalid payment details: Amount must be greater than zero. Provided: -100.0
```

Explanation

- The *read* method of *PaymentReaderService* class reads the inputs and successfully parses the date (31-07-2023) and amount (-100.00). It also reads the payer account number (1234567890), receiver account number (0987654321), and purpose of payment ("Payment for services").
- A *PaymentInfo* object is created with these details.
- The *validate* method of *PaymentValidationService* class validates the payment details:
    - The amount is not positive (-100.00 <= 0).
    - An *InvalidPaymentDetailsException* is thrown with the message "Amount must be greater than zero. Provided: -100.00"
- The resources (*PaymentReaderService* and *PaymentValidationService*) are closed with appropriate messages written to the *bufferedWriter*.
- The exception is caught in the try block in the main method, and the exception message is written to the *bufferedWriter*: "InvalidPaymentDetailsException : Amount must be greater than zero. Provided: -100.00"